Universidad Externado de Colombia
Departamento de Matemáticas
Pregrado en Ciencia de Datos

# Proyecto Integrador: Primera Entrega

## Implementación de un método Gauss-Newton libre de derivadas para mínimos cuadrados no lineales

Artículo base: A derivative-free Gauss-Newton method
(Cartis y Roberts, 2019)

**Integrantes:**
David Castro
Alejandro Pardo
Tomás Rincón

Curso: Métodos Numéricos
Docente: Edwin Fernando Muñoz

Bogotá, 2026

Este informe corresponde a la primera entrega del Proyecto Integrador del curso Métodos Numéricos. Según el microcurrículo (2026-2), se entrega en la semana 5; la segunda entrega va en la semana 10 y la entrega final en la semana 16. Durante el semestre trabajaremos con el artículo "A derivative-free Gauss-Newton method" (Cartis y Roberts, 2019), que propone DFO-GN, un método para resolver problemas de mínimos cuadrados sin usar las derivadas de las funciones. A continuación presentamos la comprensión del artículo, la formulación del problema, la fuente de información y el plan de implementación, en el orden en que el curso desarrolla los temas necesarios.

## 1. Comprensión del artículo

El artículo resuelve un problema de mínimos cuadrados: hallar el punto x que minimiza la suma de los cuadrados de varias funciones $r_1(x)$, ..., $r_m(x)$, llamadas residuos, sin conocer sus derivadas. Esta situación aparece cuando las funciones provienen de un programa externo, una simulación costosa o mediciones con ruido, casos en los que calcular las derivadas a mano, o incluso por diferencias finitas, no resulta práctico ni confiable.

La idea central de DFO-GN es la misma del Gauss-Newton clásico que se vio de forma introductoria en el curso: aproximar cada residuo por una recta alrededor del punto actual y usar esa aproximación para decidir el siguiente paso. La diferencia está en que, al no haber derivadas, la aproximación lineal se construye evaluando la función en n+1 puntos cercanos y resolviendo un sistema lineal con esos valores, que hace las veces de Jacobiano. El algoritmo avanza dentro de una región de confianza: en cada iteración se define una zona alrededor del punto actual donde se confía en la aproximación, se calcula el mejor paso dentro de esa zona y, según qué tan bien funcionó, la zona se agranda o se encoge para la siguiente iteración.

Para que el método funcione, los autores suponen dos cosas. Primero, que los residuos son suaves, es decir, que no cambian de forma brusca entre puntos cercanos. Segundo, que los puntos usados para construir la aproximación lineal están bien repartidos en el espacio, ni demasiado juntos ni alineados en una misma dirección. Si esta segunda condición falla, la aproximación deja de ser confiable y el algoritmo puede quedarse estancado.

Los autores demuestran que el método converge globalmente a un punto crítico de primer orden, donde el gradiente de la función objetivo es cero, sin importar el punto de partida. También acotan el número de iteraciones del peor caso: a lo sumo $O(\varepsilon^{-2})$ iteraciones para alcanzar una precisión ε. En las pruebas numéricas comparan DFO-GN con BOBYQA, DFBOLS y POUNDERS sobre 53 problemas estándar de mínimos cuadrados, y encuentran que necesita un número similar de evaluaciones de la función, pero corre mucho más rápido, entre 7 y más de 400 veces según el problema, porque usa menos puntos para construir cada aproximación.

Los propios autores reconocen dos limitaciones. El método pierde algo de precisión, entre 5% y 10%, cuando las evaluaciones tienen ruido, comparado con métodos que usan aproximaciones más elaboradas. Y hereda una debilidad conocida del Gauss-Newton clásico: converge más lento en problemas donde el mejor valor

posible de la función objetivo no es cero.

## 2. Formulación del problema computacional

Nuestro objetivo es implementar, con las herramientas del curso, una versión simplificada de DFO-GN y usarla para resolver un subconjunto de los problemas de prueba del artículo, comparando su comportamiento con un método de referencia que sí tiene acceso a las derivadas, el Gauss-Newton clásico.

El contexto de aplicación es la optimización sin derivadas de problemas de mínimos cuadrados. Las variables que intervienen son el punto actual $x_k$, el conjunto de n+1 puntos cercanos usado para aproximar el Jacobiano, el Jacobiano aproximado que resulta de resolver el sistema lineal y el radio de la región de confianza, que se ajusta en cada iteración según qué tan buena resultó la predicción del modelo frente al valor real de la función.

Esperamos reproducir, sobre todo, el número de evaluaciones de la función necesarias para alcanzar distintos niveles de precisión, como reporta la sección 5 del artículo. Para ello tomaremos un grupo reducido de problemas de la Tabla 2 del Apéndice C, con pocas variables (entre 2 y 6), de modo que la implementación en Python sea manejable en el tiempo del curso. No pretendemos reproducir todo el artículo, sino comprobar que lo que reportan los autores se observa también en nuestra implementación: un menor costo por iteración y un número de evaluaciones parecido al de un método con derivadas.

## 3. Obtención de la información

No usaremos datos reales, porque el problema no proviene de mediciones sino de la optimización de funciones matemáticas conocidas. La información saldrá de dos fuentes:

- Las funciones de prueba estándar del artículo (Tabla 2, Apéndice C de Cartis y Roberts, 2019), que incluyen la expresión explícita de r(x), el punto de partida $x_0$ y el valor óptimo conocido. Es la misma fuente que usan los autores para validar su método, así que resulta apropiada para comparar resultados.
- Datos simulados para las pruebas de robustez frente a ruido, generados agregando una perturbación aleatoria gaussiana a cada evaluación del residuo, según la metodología de la sección 5.2 del artículo. Fijaremos una semilla aleatoria en NumPy para que los experimentos sean reproducibles. La generación formal de números pseudoaleatorios se ve en la semana 13 del curso, antes de que necesitemos construir estas pruebas.

## 4. Plan de implementación

En esta primera fase combinamos tres métodos numéricos vistos en el curso, cada uno encargado de una parte distinta del algoritmo (Tabla 1).

**Tabla 1. Métodos numéricos previstos para la implementación**

| Etapa del algoritmo | Método numérico | Justificación |
|---|---|---|
| Aproximar el Jacobiano evaluando r(x) en n+1 puntos cercanos (ecuación 2.4 del artículo) | Eliminación de Gauss | El sistema que hay que resolver es cuadrado (n×n). Triangularizarlo y resolver por sustitución hacia atrás es lo que se vio en la sesión de eliminación de Gauss. |
| Gauss-Newton clásico con derivadas, usado como referencia | Factorización QR | Cada paso resuelve un problema de mínimos cuadrados con un Jacobiano rectangular (más filas que columnas). QR evita el mal condicionamiento de las ecuaciones normales. |
| Subproblema de región de confianza: calcular el paso $s_k$ dentro de la zona de radio $\Delta_k$ | Punto de Cauchy | Es la condición mínima que pide el artículo para asegurar que el método avance. Se calcula con una fórmula directa, sin iterar. |

Los criterios para comparar nuestra implementación con el método de referencia con derivadas son los siguientes:

- Número de evaluaciones de la función necesarias para alcanzar una tolerancia dada, siguiendo la métrica del artículo.
- Estabilidad frente a ruido: repetiremos el experimento con y sin ruido gaussiano en las evaluaciones de la función.
- Costo computacional: tiempo de ejecución y tamaño del sistema lineal que se resuelve en cada iteración, frente a lo que exigiría un método con Jacobiano exacto.

Este plan sigue el cronograma del curso (2026-2). El álgebra lineal numérica (Gauss, LU, Jacobi y Gauss-Seidel, mínimos cuadrados y QR) se ve entre las semanas 6 y 8, así que al implementar ya habremos visto eliminación de Gauss y QR. Los métodos para ecuaciones no lineales (semana 9) quedan como apoyo por si se mejora el subproblema de región de confianza más allá del punto de Cauchy. Los temas que aún no se han visto se repasarán antes de la clase correspondiente.

Estos criterios se retomarán en la segunda entrega (semana 10), con la comparación numérica completa entre DFO-GN y el Gauss-Newton clásico. Para la entrega final (semana 16), la introducción a la optimización numérica (semana 14) dará el lenguaje para explicar mejor el comportamiento de DFO-GN.

## Referencias

Cartis, C., & Roberts, L. (2019). A derivative-free Gauss-Newton method. Mathematical Programming Computation, 11, 631-674.